

Autenticação e Autorização com Spring Security, JWT Tokens e Roles

Professor: João Keslei



Project

☐ Gradle - Groovy ☐ Gradle - Kotlin ☒ **Java** ☐ Kotlin ☐ Groovy
☒ **Maven**

Spring Boot

☐ 4.0.0 (SNAPSHOT) ☐ 4.0.0 (M1) ☐ 3.5.5 (SNAPSHOT) ☒ **3.5.4**
☐ 3.4.9 (SNAPSHOT) ☐ 3.4.8

Project Metadata

Group

Artifact

Name

Description

Package name

Packaging ☒ **Jar** ☐ War

Java ☐ 24 ☐ 21 ☒ **17**

Language

Dependencies

ADD DEPENDENCIES... CTRL + B

Spring Web **WEB**

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

Spring Data JPA **SQL**

Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.

H2 Database **SQL**

Provides a fast in-memory database that supports JDBC API and R2DBC access, with a small (2mb) footprint. Supports embedded and server modes as well as a browser based console application.

Spring Security **SECURITY**

Highly customizable authentication and access-control framework for Spring applications.

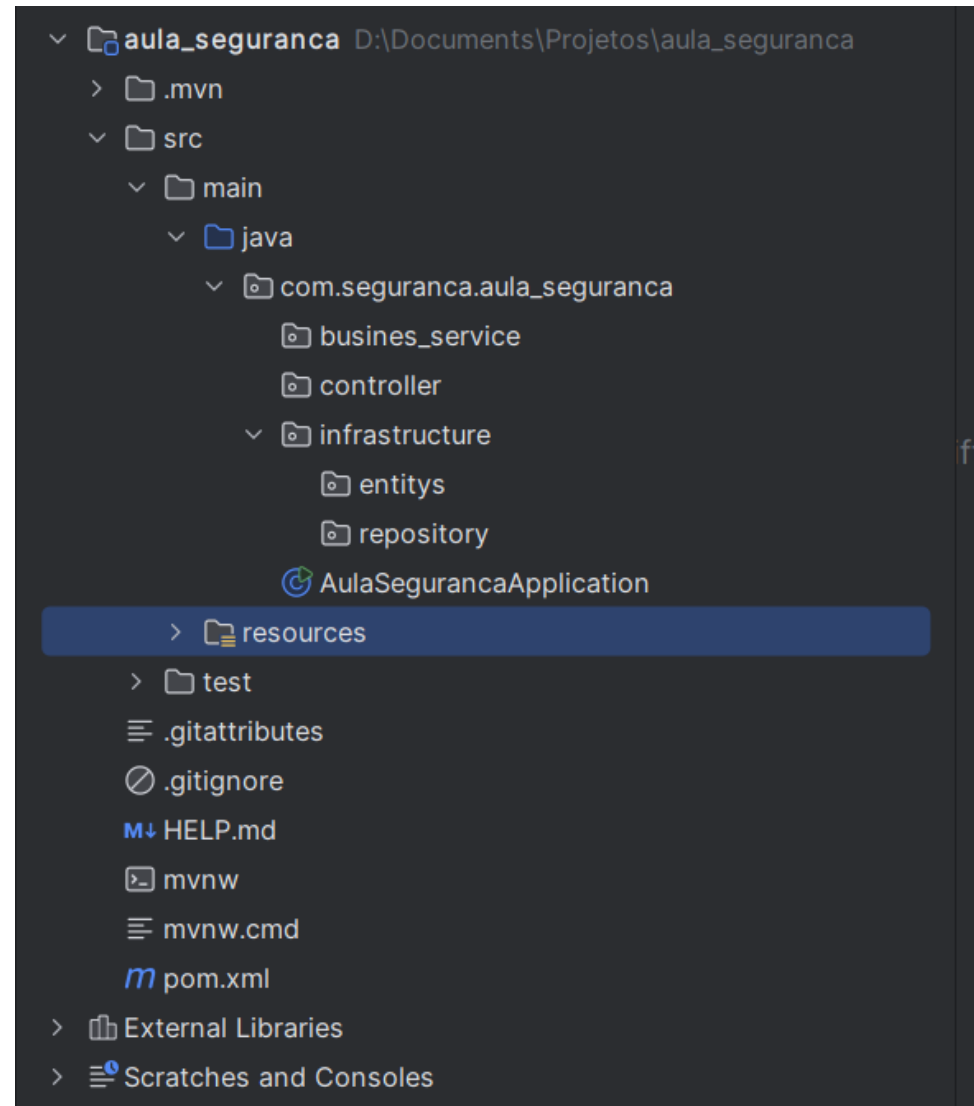
Validation **I/O**

Bean Validation with Hibernate validator.

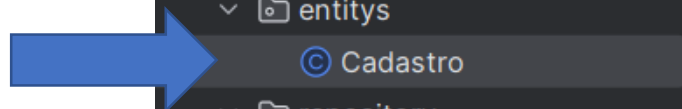
Lombok **DEVELOPER TOOLS**

Java annotation library which helps to reduce boilerplate code.

Organização por camadas



Classe principal, criado em entitys com o nome de Cadastro.



```
1 package com.seguranca.aula_seguranca.infrastructure.entitys;
2
3 import jakarta.persistence.*;
4 import lombok.*;
5
6 @Getter 14 usages
7 @Setter
8 @AllArgsConstructor
9 @NoArgsConstructor
10 @Builder
11 @Table(name = "cadastro")
12 @Entity
13 public class Cadastro {
14
15     @Id
16     @GeneratedValue(strategy = GenerationType.AUTO)
17     private Long id;
18
19     @Column(name = "nome")
20     private String nome;
21
22     @Column(name = "idade")
23     private Integer idade;
24 }
25
```

1 Anotações do Lombok

java

 Copiar  Editar

```
@Getter  
@Setter  
@AllArgsConstructor  
@NoArgsConstructor  
@Builder
```

- `@Getter` → cria **automaticamente métodos get** para todos os atributos (`getId()` , `getNome()` , `getIdade()`).
- `@Setter` → cria **automaticamente métodos set** para todos os atributos (`setId()` , `setNome()` , `setIdade()`).
- `@AllArgsConstructor` → cria **construtor com todos os parâmetros** (`Cadastro(Long id, String nome, Integer idade)`).
- `@NoArgsConstructor` → cria **construtor vazio** (`Cadastro()`).
- `@Builder` → permite criar objetos usando o padrão **Builder**, ex:

java

 Copiar  Editar

```
Cadastro c = Cadastro.builder()  
    .nome("João")  
    .idade(36)  
    .build();
```

2 Anotações JPA / Hibernate

java

 Copiar

```
@Entity  
@Table(name = "cadastro")
```

- `@Entity` → indica que a classe representa uma **tabela no banco de dados**.
- `@Table(name = "cadastro")` → define o **nome da tabela** no banco (nesse caso, "cadastro").

3 Atributos

java

 Copiar

```
@Id
@GeneratedValue(strategy = GenerationType.AUTO)
private Long id;
```

- `@Id` → marca **chave primária** da tabela.
- `@GeneratedValue(strategy = GenerationType.AUTO)` → o banco **gera o valor automaticamente** (incremento automático).

java

```
@Column(name = "nome")
private String nome;

@Column(name = "idade")
private Integer idade;
```

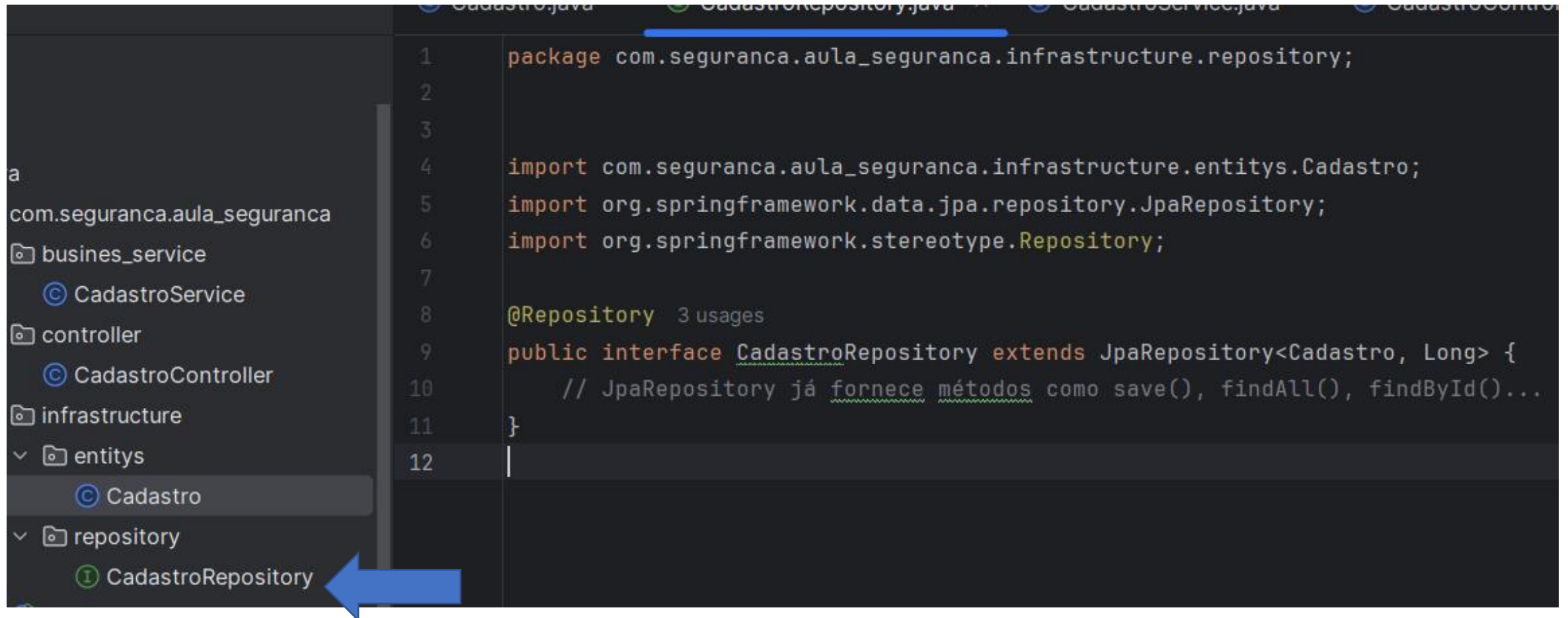
- `@Column(name = "...")` → define o nome da coluna no banco.
- `String nome` → armazena o nome da pessoa.
- `Integer idade` → armazena a idade da pessoa.

◆ Resumo simples

Essa classe representa **uma tabela de cadastro de pessoas**:

- Cada pessoa tem `id`, `nome` e `idade`.
- O banco cria automaticamente o `id`.
- O Lombok cria **getters, setters, construtores e builder** automaticamente.
- A tabela no banco se chama **"cadastro"**.

Depois criar a Interface



The screenshot shows an IDE with a project structure on the left and a code editor on the right. The project structure includes a package `com.seguranca.aula_seguranca` with sub-packages `busines_service`, `controller`, `infrastructure`, `entitys`, and `repository`. The `entitys` package contains `Cadastro`, and the `repository` package contains `CadastroRepository`. The code editor shows the implementation of `CadastroRepository.java` in the `infrastructure.repository` package. It imports `Cadastro`, `JpaRepository`, and `Repository`. The interface `CadastroRepository` extends `JpaRepository<Cadastro, Long>` and includes a comment about the methods provided by `JpaRepository`.

```
1 package com.seguranca.aula_seguranca.infrastructure.repository;
2
3
4 import com.seguranca.aula_seguranca.infrastructure.entitys.Cadastro;
5 import org.springframework.data.jpa.repository.JpaRepository;
6 import org.springframework.stereotype.Repository;
7
8 @Repository 3 usages
9 public interface CadastroRepository extends JpaRepository<Cadastro, Long> {
10     // JpaRepository já fornece métodos como save(), findAll(), findById()...
11 }
12 |
```



```
@Repository
```

```
public interface CadastroRepository extends JpaRepository<Cadastro, Long> {  
    // JpaRepository já fornece métodos como save(), findAll(), findById()...  
}
```

1 Anotação @Repository

- Indica que esta interface é **um repositório de dados**.
- O Spring reconhece que ela será usada para **acessar o banco de dados**.
- Também permite que o Spring trate **exceções de banco** automaticamente.

2 Extensão de JpaRepository

java

```
extends JpaRepository<Cadastro, Long>
```

- JpaRepository é do Spring Data JPA.
- Ela fornece **métodos prontos** para operar no banco sem escrever SQL:

Método	O que faz
save()	Salva um objeto no banco
findAll()	Retorna todos os registros
findById(id)	Busca um registro pelo ID
deleteById(id)	Deleta um registro pelo ID
count()	Conta quantos registros existem

- <Cadastro, Long> indica:
 - Cadastro → tipo da entidade (a tabela que você criou).
 - Long → tipo da chave primária (id).

3 Benefício

Você **não precisa escrever SQL ou métodos básicos** para salvar, listar ou deletar cadastros.

Se depois quiser, pode adicionar métodos personalizados, por exemplo:

```
java
```

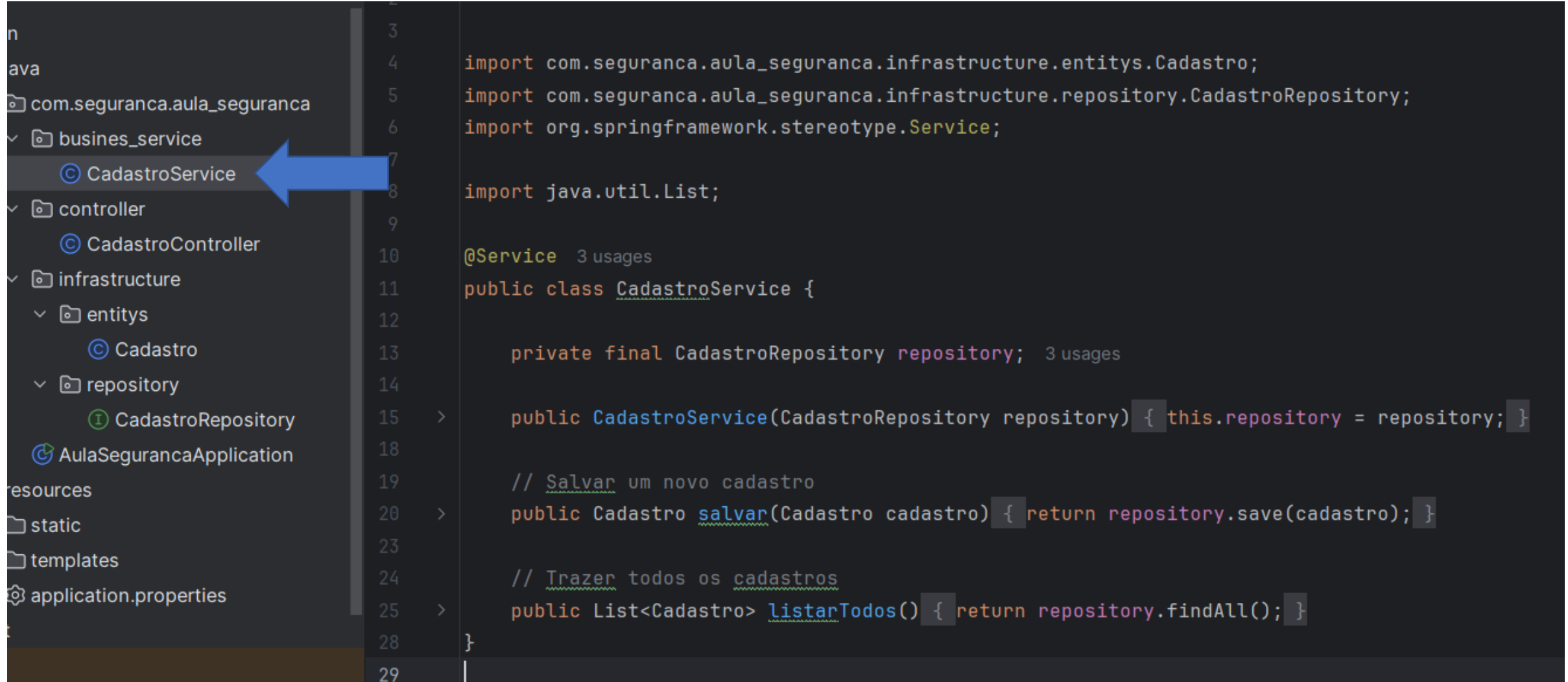
```
List<Cadastro> findByName(String nome);
```

O Spring Data JPA **gera a query automaticamente** com base no nome do método.

◆ Resumo rápido

- Essa interface conecta sua classe `Cadastro` ao banco.
- Herda métodos prontos do `JpaRepository`.
- Facilita salvar, listar, buscar e deletar cadastros sem escrever código extra.

CadastroService



The image shows an IDE with a package explorer on the left and a code editor on the right. The package explorer shows the following structure:

- com.seguranca.aula_seguranca
 - business_service
 - CadastroService** (highlighted with a blue arrow)
 - controller
 - CadastroController
 - infrastructure
 - entitys
 - Cadastro
 - repository
 - CadastroRepository
 - AulaSegurancaApplication
- resources
 - static
 - templates
 - application.properties

The code editor shows the following code for `CadastroService`:

```
3
4 import com.seguranca.aula_seguranca.infrastructure.entitys.Cadastro;
5 import com.seguranca.aula_seguranca.infrastructure.repository.CadastroRepository;
6 import org.springframework.stereotype.Service;
7
8 import java.util.List;
9
10 @Service 3 usages
11 public class CadastroService {
12
13     private final CadastroRepository repository; 3 usages
14
15     > public CadastroService(CadastroRepository repository) { this.repository = repository; }
16
17
18     // Salvar um novo cadastro
19
20     > public Cadastro salvar(Cadastro cadastro) { return repository.save(cadastro); }
21
22
23     // Trazer todos os cadastros
24
25     > public List<Cadastro> listarTodos() { return repository.findAll(); }
26
27
28 }
29
```

1 @Service

- Indica que essa classe é um **serviço** do Spring.
 - Serviços são responsáveis pela **lógica de negócio**, ou seja, regras ou operações que usam os dados do banco.
 - O Spring consegue **injetar esse serviço** em controllers ou outros serviços.
-

2 Construtor com CadastroRepository

java

 Copiar  Editar

```
public CadastroService(CadastroRepository repository) {  
    this.repository = repository;  
}
```

- Recebe o `CadastroRepository` do Spring via **injeção de dependência**.
- Isso permite que o serviço acesse o banco sem precisar criar manualmente conexões ou queries SQL.

3 Métodos do Service

a) salvar(Cadastro cadastro)

java

```
return repository.save(cadastro);
```

- Salva o objeto `Cadastro` no banco de dados.
- Se o cadastro **não existir**, cria um novo registro.
- Se já existir (mesmo `id`), atualiza os dados.

b) listarTodos()

java

```
return repository.findAll();
```

- Retorna **todos os cadastros** do banco como uma lista (`List<Cadastro>`).
- Usa o método pronto do `JpaRepository`, então você não precisa escrever SQL.

◆ Resumo rápido

- `CadastroService` é a **camada de lógica de negócio**.
- Ele usa o `CadastroRepository` para falar com o banco.
- Tem métodos **simples**: salvar um cadastro e listar todos os cadastros.
- Facilita separar responsabilidades: Controller cuida da web, Service cuida da lógica, Repository cuida do banco.

CadastroController

```
4 import com.seguranca.aula_seguranca.busines_service.CadastroService;
5 import com.seguranca.aula_seguranca.infrastructure.entitys.Cadastro;
6 import org.springframework.http.ResponseEntity;
7 import org.springframework.web.bind.annotation.*;
8
9 import java.util.List;
10 import org.springframework.web.bind.annotation.*;
11
12 @RestController no usages
13 @RequestMapping("/cadastros")
14 public class CadastroController {
15
16     private final CadastroService cadastroService; // apenas uma variável 4 usages
17
18     // Construtor para injeção de dependência
19     public CadastroController(CadastroService cadastroService) { no usages
20     |     this.cadastroService = cadastroService;
21     | }
22
23     // Endpoint POST para salvar um novo cadastro via JSON
24     @PostMapping no usages
25     public ResponseEntity<Cadastro> salvar(@RequestBody Cadastro cadastro) {
26     |     Cadastro salvo = cadastroService.salvar(cadastro);
27     |     return ResponseEntity.ok(salvo);
28     | }
```



```
// Endpoint GET para listar todos os cadastros
@GetMapping no usages
public ResponseEntity<List<Cadastro>> listarTodos() {
    List<Cadastro> lista = cadastroService.listarTodos();
    return ResponseEntity.ok(lista);
}

// Endpoint GET para cadastro rápido via URL (para navegador)
@GetMapping("/add") no usages
public String addCadastro(@RequestParam String nome,
                           @RequestParam Integer idade) {
    Cadastro c = new Cadastro();
    c.setNome(nome);
    c.setIdade(idade);
    cadastroService.salvar(c);
    return "Cadastro salvo: " + c.getNome() + ", idade: " + c.getIdade();
}
```

1 @RestController

- Indica que esta classe é um **controlador REST**, ou seja, vai receber requisições HTTP e retornar respostas JSON ou texto.
-

2 @RequestMapping("/cadastros")

- Define que **todos os endpoints da classe começam com** `/cadastros`.
- Exemplo:
 - `GET /cadastros` → listar todos
 - `POST /cadastros` → criar novo cadastro
 - `GET /cadastros/add` → cadastro rápido via URL

3 CadastroService

java

 Copiar

```
private final CadastroService cadastroService;

public CadastroController(CadastroService cadastroService) {
    this.cadastroService = cadastroService;
}
```

- Injeta o **service** no controller, permitindo chamar métodos que acessam o banco de dados.
- **final** garante que a variável não será modificada depois de inicializada.

4 Endpoints

a) POST /cadastros

java

`@PostMapping`

```
public ResponseEntity<Cadastro> salvar(@RequestBody Cadastro cadastro)
```

- Recebe um objeto `Cadastro` em **JSON**.
- Chama `cadastroService.salvar(cadastro)` para salvar no banco.
- Retorna o cadastro salvo em JSON.

Exemplo JSON para enviar via Postman:

json

```
{  
  "nome": "João",  
  "idade": 36  
}
```

b) GET `/cadastros`

```
java
```

```
@GetMapping
```

```
public ResponseEntity<List<Cadastro>> listarTodos()
```

- Lista **todos os cadastros** do banco.
- Retorna uma lista JSON.

c) GET `/cadastros/add`

java

```
@GetMapping("/add")  
public String addCadastro(@RequestParam String nome,  
                           @RequestParam Integer idade)
```

- Permite cadastro **direto via navegador**:

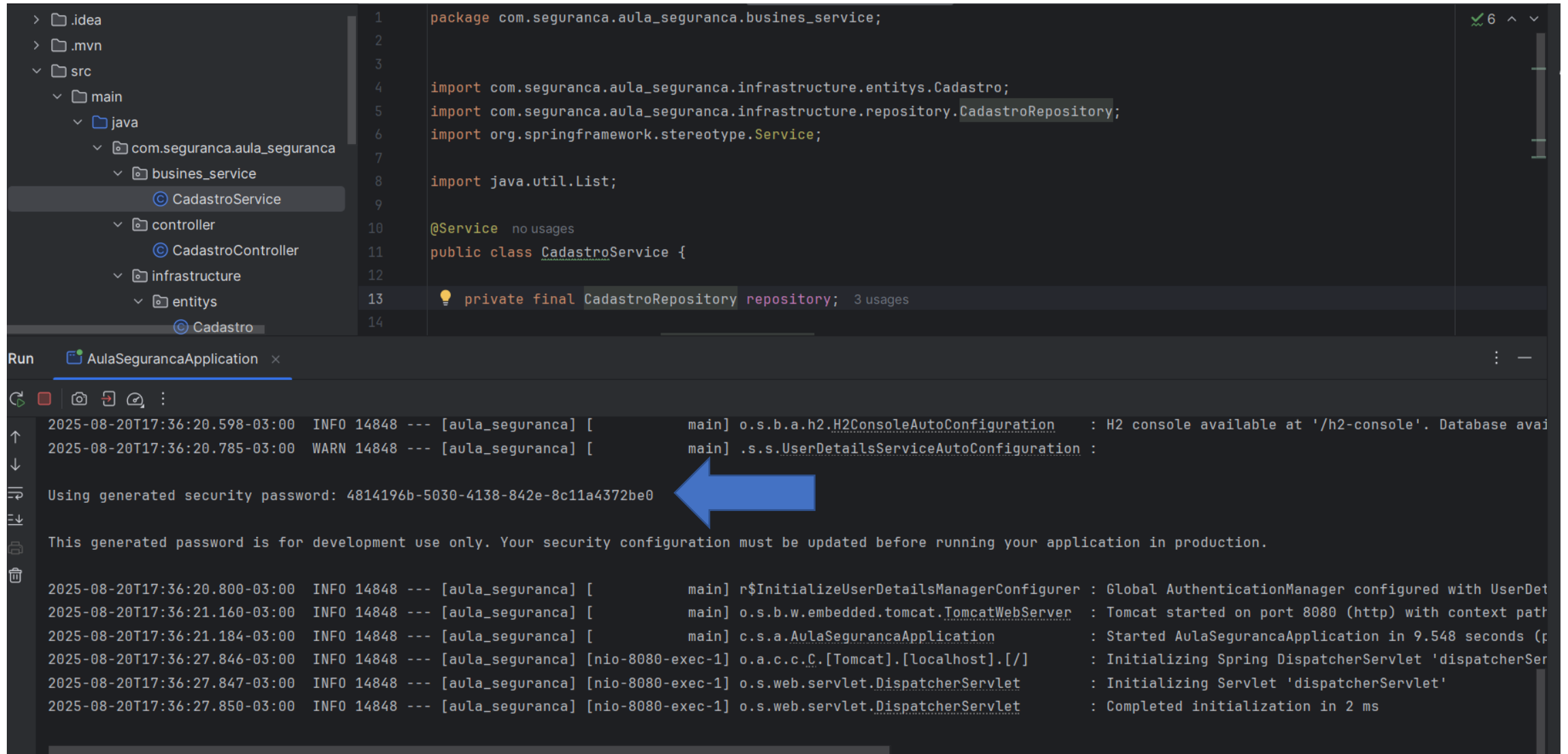
bash

```
http://localhost:8080/cadastros/add?nome=Joao&idade=36
```

- Cria um objeto `Cadastro`, salva no banco e retorna uma mensagem simples.

Esse endpoint é útil para testes rápidos, sem precisar de Postman.

Onde é gerado a senha



```
package com.seguranca.aula_seguranca.busines_service;

import com.seguranca.aula_seguranca.infrastructure.entitys.Cadastro;
import com.seguranca.aula_seguranca.infrastructure.repository.CadastroRepository;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class CadastroService {

    private final CadastroRepository repository;

    ...

}
```

Run AulaSegurancaApplication

```
2025-08-20T17:36:20.598-03:00 INFO 14848 --- [aula_seguranca] [main] o.s.b.a.h2.H2ConsoleAutoConfiguration : H2 console available at '/h2-console'. Database available
2025-08-20T17:36:20.785-03:00 WARN 14848 --- [aula_seguranca] [main] .s.s.UserDetailsServiceAutoConfiguration :

Using generated security password: 4814196b-5030-4138-842e-8c11a4372be0

This generated password is for development use only. Your security configuration must be updated before running your application in production.

2025-08-20T17:36:20.800-03:00 INFO 14848 --- [aula_seguranca] [main] r$InitializeUserDetailsManagerConfigurer : Global AuthenticationManager configured with UserDetailsService
2025-08-20T17:36:21.160-03:00 INFO 14848 --- [aula_seguranca] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2025-08-20T17:36:21.184-03:00 INFO 14848 --- [aula_seguranca] [main] c.s.a.AulaSegurancaApplication : Started AulaSegurancaApplication in 9.548 seconds (process running for 10.07)
2025-08-20T17:36:27.846-03:00 INFO 14848 --- [aula_seguranca] [nio-8080-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2025-08-20T17:36:27.847-03:00 INFO 14848 --- [aula_seguranca] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2025-08-20T17:36:27.850-03:00 INFO 14848 --- [aula_seguranca] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 2 ms
```

Localhost:8080

Vai ser direcionado automaticamente para a tela do login



localhost:8080/login

YouTube FINAL FANTASY XIV... Cursos - Curso em... Hora da Aventura [...] D&D Beyond - An o... (6208) Git and GitH... Seu perfil GitHub SIA - Sistema de Inf...

Please sign in

Padrao/default →

user

.....

Sign in



localhost:8080/cadastros/add?nome=Joao&idade=36



Gmail



YouTube

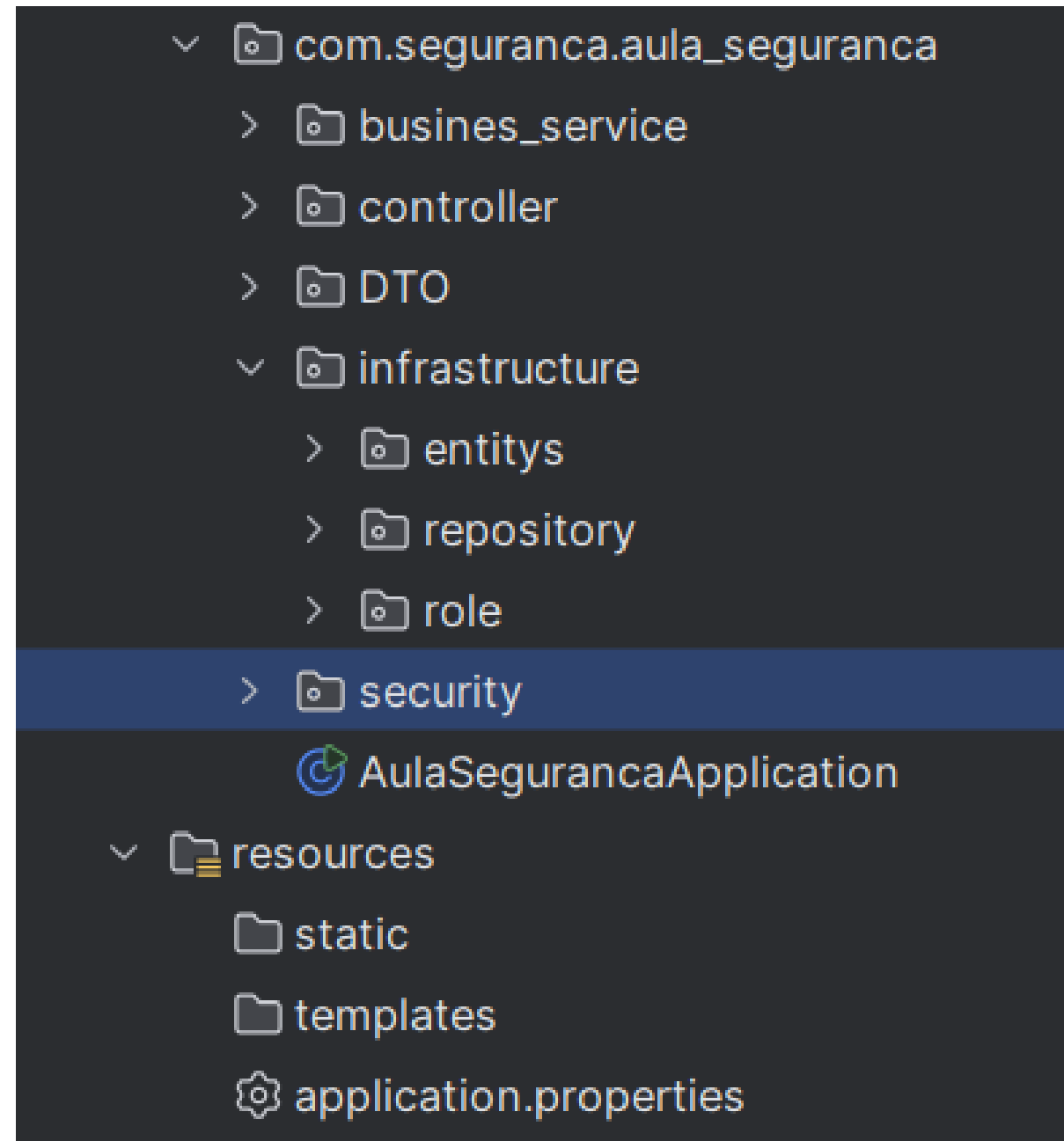
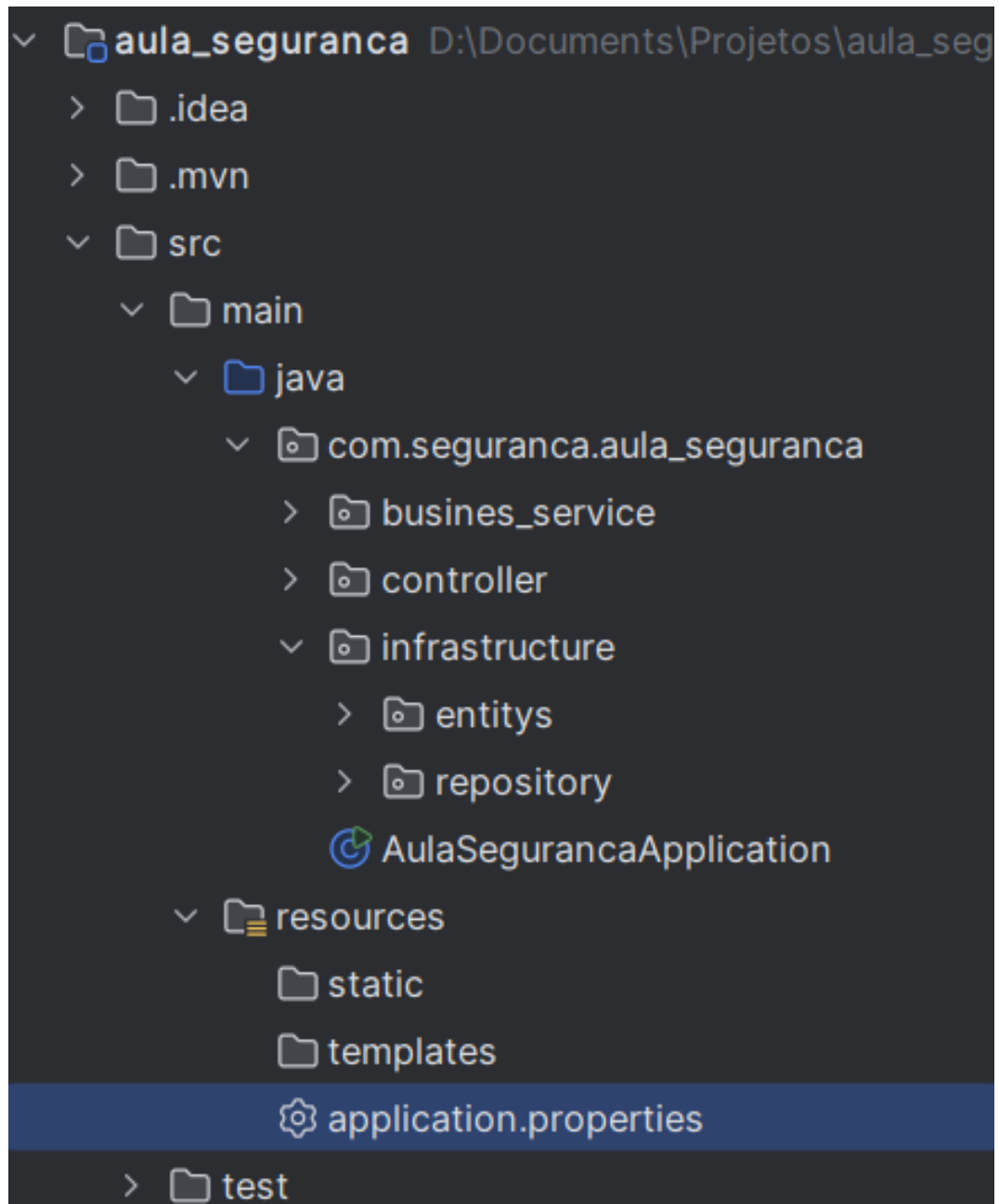


FINAL FANTASY XIV...

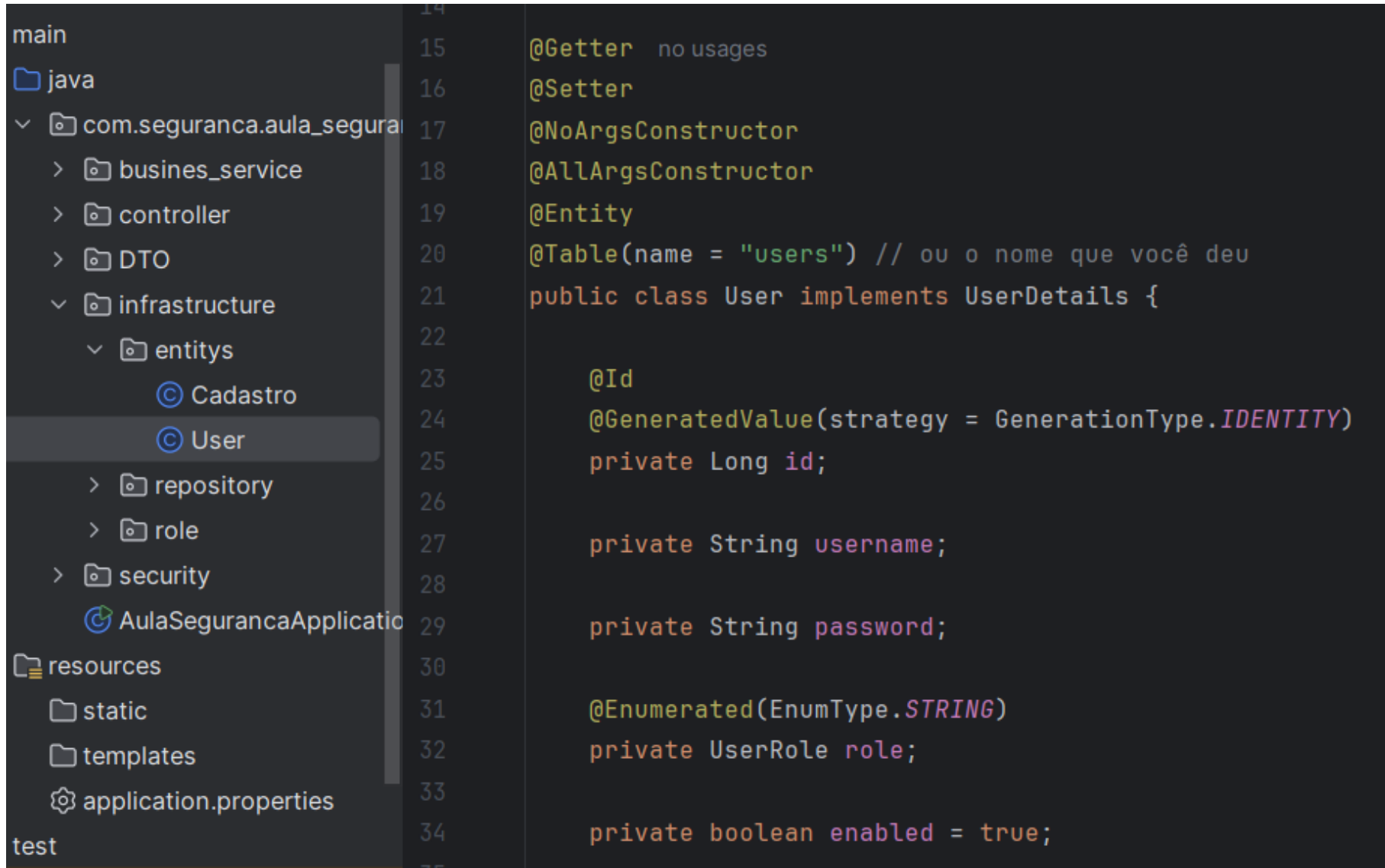


Cursos - Curso em...

Cadastro salvo: Joao, idade: 36



Começo da Segurança



The image shows a screenshot of an IDE with a project structure on the left and a Java class definition on the right.

Project Structure (Left Panel):

- main
 - java
 - com.seguranca.aula_seguranca
 - business_service
 - controller
 - DTO
 - infrastructure
 - entitys
 - Cadastro
 - User**
 - repository
 - role
 - security
 - AulaSegurancaApplication
 - resources
 - static
 - templates
 - application.properties
 - test

Java Class Definition (Right Panel):

```
14
15 @Getter no usages
16 @Setter
17 @NoArgsConstructor
18 @AllArgsConstructor
19 @Entity
20 @Table(name = "users") // ou o nome que você deu
21 public class User implements UserDetails {
22
23     @Id
24     @GeneratedValue(strategy = GenerationType.IDENTITY)
25     private Long id;
26
27     private String username;
28
29     private String password;
30
31     @Enumerated(EnumType.STRING)
32     private UserRole role;
33
34     private boolean enabled = true;
35 }
```

```
public class User implements UserDetails {
```

```
// ♦ Construtor customizado para facilitar a criação de novos usuários
```

```
✓  public User(String username, String password, UserRole role) { 1 usage
```

```
    this.username = username;
```

```
    this.password = password;
```

```
    this.role = role;
```

```
}
```

```
// Implementações obrigatórias do UserDetails (já passamos antes)
```

```
@Override no usages
```

```
✓ public Collection<? extends GrantedAuthority> getAuthorities() {
```

```
    return List.of(new SimpleGrantedAuthority(role.name()));
```

```
}
```

```
@Override no usages
```

```
public boolean isAccountNonExpired() { return true; }
```

```
@Override no usages
```

```
public boolean isAccountNonLocked() { return true; }
```

```
@Override no usages
```

```
public boolean isCredentialsNonExpired() { return true; }
```

```
@Override
```

```
public boolean isEnabled() { return enabled; }
```

```
}
```

📌 Classe `User` como entidade do banco + implementação do `UserDetails`

Esse código está mapeando uma **entidade JPA** (usuário que será salvo no banco) e, ao mesmo tempo, está **integrando com o Spring Security** para que esse usuário seja usado na autenticação/autorização.

```
private String username;  
private String password;
```

- **username** → nome de login do usuário.
- **password** → senha criptografada (não deve ser armazenada em texto puro).

java

📄 Copiar ✎ Editar

```
@Enumerated(EnumType.STRING)  
private UserRole role;
```

- `@Enumerated(EnumType.STRING)` → diz que o **enum** `UserRole` será armazenado como **texto** no banco (ex.: `"ADMIN"`, `"USER"`), em vez de número.
- `role` → representa o perfil/função do usuário (ADMIN, USER, etc).

```
private boolean enabled = true;
```

- Indica se o usuário está **ativo** ou **desativado**.
- Spring Security usa esse campo no método `isEnabled()`.

◆ Construtor customizado

java

 Copiar  Editar

```
public User(String username, String password, UserRole role) {  
    this.username = username;  
    this.password = password;  
    this.role = role;  
}
```

- Facilita criar novos usuários sem precisar passar o `id` ou `enabled` (que já tem valor padrão `true`).
- Exemplo de uso:

java

 Copiar  Editar

```
User user = new User("joao", "1234", UserRole.ADMIN);
```

◆ Métodos do UserDetails

O `User` implementa `UserDetails`, que é a interface do Spring Security usada para representar um usuário autenticável.

Isso faz com que o Spring consiga entender como carregar os dados do usuário do banco.

```
@Override
public Collection<? extends GrantedAuthority> getAuthorities() {
    return List.of(new SimpleGrantedAuthority(role.name()));
}
```

- Retorna as **autoridades/roles** do usuário.
- `new SimpleGrantedAuthority(role.name())` → transforma o enum (`ADMIN`, `USER`) em uma role reconhecida pelo Spring Security.
 - Exemplo: `"ROLE_ADMIN"`, `"ROLE_USER"`.

```
@Override
public boolean isAccountNonExpired() { return true; }

@Override
public boolean isAccountNonLocked() { return true; }

@Override
public boolean isCredentialsNonExpired() { return true; }

@Override
public boolean isEnabled() { return enabled; }
```

Esses métodos dizem se a conta do usuário está válida para login:

- `isAccountNonExpired` → conta não expirou.
- `isAccountNonLocked` → conta não está bloqueada.
- `isCredentialsNonExpired` → senha não está expirada.
- `isEnabled` → usuário está ativo (usa o campo `enabled`).

No código atual, todos retornam `true` (ou usam o campo `enabled`), ou seja, **todos os usuários estão sempre válidos**.

Resumindo

Esse código:

1. Define um **usuário no banco** (entidade JPA).
2. Implementa `UserDetails`, permitindo que o Spring Security use esses dados para autenticação.
3. Garante que cada usuário tem:
 - `id`, `username`, `password`, `role`, `enabled`.
 - E que o Spring Security sabe interpretar esses dados.

m.seguranca.aula_segura

busines_service

controller

DTO

infrastructure

entitys

Ⓢ Cadastro

Ⓢ User

repository

role

Ⓢ UserRole

security

AulaSegurancaApplicatio

ces

tic

nplates

application.properties

2

> import ...

5

6

public enum UserRole { 3 usages

7

8

ADMIN("admin"), no usages

9

USER("user"); no usages

10

11

private String role; 3 usages

12

13

> UserRole(String role) { this.role = role; }

16

17

@JsonValue // quando for serializar, usa este valor no usages

18

> public String getRole() { return role; }

21

22

@JsonCreator // quando receber JSON, converte para o enum certo no usages

23

@

public static UserRole fromValue(String value) {

24

for (UserRole role : UserRole.values()) {

25

if (role.role.equalsIgnoreCase(value)) {

26



return role;

27

}

28

}

29

throw new IllegalArgumentException("Invalid role: " + value);

30

}

31

}

Boa! 🙌 Esse `enum UserRole` está configurado para funcionar **tanto na entrada (request JSON)** quanto na saída (response JSON).

◆ Como ele funciona na prática

1. Quando você salvar um usuário no banco via Postman (POST):

- O campo `role` deve ser enviado como **string** (`"admin"` ou `"user"`) e o Jackson vai converter automaticamente para o `enum`.

Exemplo de JSON no Postman:

json

📋 Copiar ✎ Editar

```
{
  "nome": "João",
  "email": "joao@email.com",
  "telefone": "61999999999",
  "tipo_usuario": "admin"
}
```

👉 O `@JsonCreator` pega `"admin"` e converte para `UserRole.ADMIN`.

2. Quando você buscar esse usuário (GET):

- No retorno, o `enum` será exibido como string (`"admin"` ou `"user"`), porque o `@JsonValue` força o JSON a mostrar só o valor.

Exemplo de resposta da API:

json

 Copiar  Editar

```
{
  "id": 1,
  "nome": "João",
  "email": "joao@email.com",
  "telefone": "61999999999",
  "tipo_usuario": "admin"
}
```

👉 Note que não aparece `ADMIN`, mas sim `"admin"` (em minúsculo, porque você definiu no construtor).

◆ Testando com URL (GET no Postman ou navegador)

Se você tiver um endpoint, por exemplo:

```
java
```

```
@GetMapping("/usuarios/{id}")  
public ResponseEntity<Usuario> buscarPorId(@PathVariable Long id) {  
    return ResponseEntity.ok(usuarioService.buscarPorId(id));  
}
```

E tiver no banco o usuário com `tipo_usuario = admin`, ao acessar:

```
http://localhost:8080/usuarios/1
```

Você receberá:

```
json
```

```
{  
  "id": 1,  
  "nome": "João",  
  "email": "joao@email.com",  
  "telefone": "619999999999",  
  "tipo_usuario": "admin"  
}
```

👉 Resumindo:

- **POST /usuarios** → você manda "admin" ou "user" no JSON.
- **GET /usuarios/{id}** → você recebe "admin" ou "user" no JSON.

java

com.seguranca.aula_seguranca

> empres_service

> controller

> DTO

> infrastructure

> entitys

> Cadastro

> User

> repository

> CadastroRepository

> UserRepository

3

4

5

6

7

8

9

10

11

import com.seguranca.aula_seguranca.infrastructure.entitys.User;

import org.springframework.data.jpa.repository.JpaRepository;

import org.springframework.stereotype.Repository;

@Repository 6 usages

public interface UserRepository extends JpaRepository<User, Long> {

User findByUsername(String username); 2 usages

}

O que acontece aqui:

1. `extends JpaRepository<User, Long>`

- O Spring já cria automaticamente todos os métodos básicos de CRUD (salvar, atualizar, excluir, buscar por id, listar todos, etc.).

2. `User findByUsername(String username)`

- Esse método **não precisa ser implementado**.
- O Spring Data JPA lê o nome do método (`findByUsername`) e entende que precisa gerar a query:

sql

 Copiar  Editar

```
SELECT * FROM users WHERE username = :username
```

- Ou seja, ele retorna o usuário que tem o `username` informado.

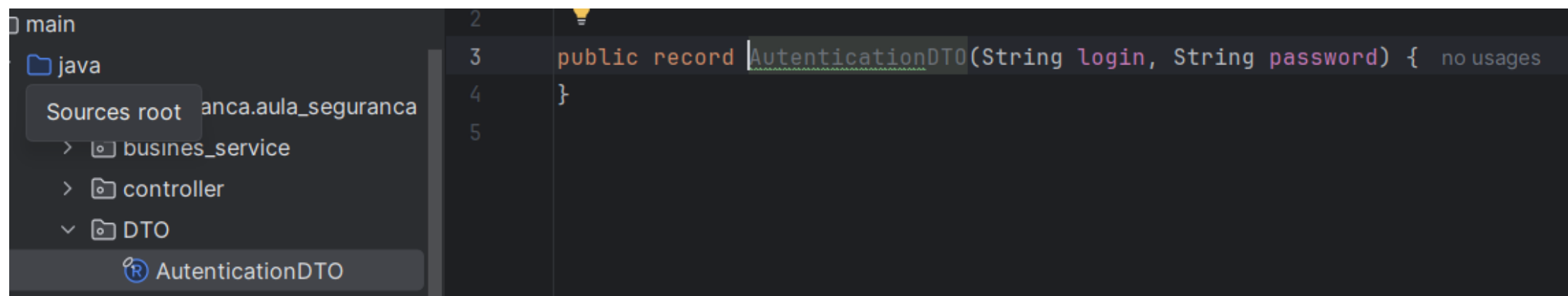
3. O `@Repository` é opcional aqui, porque o **Spring já detecta interfaces que estendem** `JpaRepository`, mas pode deixar para ficar mais explícito.

Como isso é usado?

Esse repositório é essencial no **processo de login/autenticação**.

No Spring Security, geralmente você tem um `UserDetailsService` parecido com este:

 Quando alguém tenta logar, o **Spring Security** chama esse serviço, que usa o `UserRepository` para buscar o usuário no banco.



```
2  
3 public record AuthenticationDTO(String login, String password) { no usages  
4  
5 }
```

The screenshot shows an IDE with a project structure on the left and a code editor on the right. The project structure includes a 'main' directory with a 'java' subdirectory. Inside 'java', there is a 'Sources root' directory, followed by 'anca.aula_seguranca', 'busines_service', 'controller', and 'DTO'. The 'AuthenticationDTO' class is selected in the 'DTO' directory. The code editor shows the definition of the 'AuthenticationDTO' record class, which is a public record with two parameters: 'login' and 'password', both of type 'String'. The class is named 'AuthenticationDTO' and has a 'no usages' status.

Esse `record` que você criou é um **DTO (Data Transfer Object)**, usado para transportar os dados de login do usuário (login e senha) no momento da autenticação.

👉 Em **Java 16+**, `record` é uma forma mais simples e imutável de criar classes de transporte de dados, sem precisar escrever `getters`, `setters`, `equals`, `hashCode` e `toString`.

O que ele faz:

java

```
public record AutenticationDTO(String login, String password) {  
}
```

- Cria automaticamente:
 - `private final String login;`
 - `private final String password;`
 - Construtor `public AutenticationDTO(String login, String password)`
 - `public String login()`
 - `public String password()`
 - `equals`, `hashCode` e `toString`.

Como seria usado:

📌 Num **endpoint de login**, você poderia recebê-lo assim:

java

```
@RestController
@RequestMapping("/auth")
public class AuthenticationController {

    @PostMapping("/login")
    public ResponseEntity<String> login(@RequestBody AuthenticationDTO dto) {
        // Aqui você teria lógica para autenticar o usuário
        if ("joao".equals(dto.login()) && "123".equals(dto.password())) {
            return ResponseEntity.ok("Login realizado com sucesso!");
        }
        return ResponseEntity.status(401).body("Credenciais inválidas!");
    }
}
```

Exemplo de requisição no Postman:

- Método: POST
- URL: `http://localhost:8080/auth/login`
- Body (JSON):

json

```
{  
  "login": "joao",  
  "password": "123"  
}
```

✓ O Spring converte esse JSON automaticamente em um `AuthenticationDTO`.

java

com.seguranca.aula_seguranca

>

business_service

>

controller

>

DTO

AutenticationDTO

RegisterDTO

3

4

5

6

7

8

9

10

import com.seguranca.aula_seguranca.infrastructure.role.UserRole;

public record RegisterDTO(no usages

String login, no usages

String password, no usages

UserRole role no usages

) {}

Esse `RegisterDTO` é um **Data Transfer Object (DTO)** usado quando você quiser **registrar um novo usuário**. Ele carrega os dados básicos que você precisa no momento do cadastro:

- `login` → o nome de usuário (ex: `"joao"`).
- `password` → a senha (ex: `"123456"`).
- `role` → o perfil do usuário, vindo do `enum UserRole` (pode ser `"admin"` ou `"user"`).

Exemplo de uso no Postman

Se você tiver um **endpoint de registro** no seu `Controller` (por exemplo, `@PostMapping("/auth/register")`), o corpo da requisição no Postman (em JSON) seria assim:

json

 Copiar  Editar

```
{
  "login": "joao",
  "password": "123456",
  "role": "user"
}
```


ou, se for administrador:

json

 Copiar  Editar

```
{
  "login": "maria",
  "password": "senhaSegura",
  "role": "admin"
}
```

O que acontece no backend?

- O Spring pega esse JSON,
- Converte para um objeto `RegisterDTO`,
- E você usa esse objeto no seu service/controller para criar um `User` e salvar no banco de dados.

👉 Quer que eu te mostre como ficaria o **controller** que usa esse `RegisterDTO` para registrar usuários no banco?

ain
java
com.seguranca.aula_seguranca
└─ business_service
 ─ AuthorizationService
 ─ CadastroService
controller
DTO
 ─ AutenticationDTO
 ─ RegisterDTO
infrastructure
 ─ entitys
 ─ Cadastro
 ─ User
 ─ repository
 ─ CadastroRepository
 ─ UserRepository
 ─ role
 ─ UserRole
security
AulaSegurancaApplication
resources
static

```
8
9  @Service no usages
10 public class AuthorizationService {
11
12     private final UserRepository repository; 4 usages
13
14     public AuthorizationService(UserRepository repository) { this.repository = repository; }
15
16
17
18     public boolean userExists(String login) { 1 usage
19         return repository.findByUsername(login) != null; // usar o mesmo método que declarou
20     }
21
22     public void registerUser(String login, String password, String roleStr) { no usages
23         if (userExists(login)) {
24             throw new RuntimeException("Usuário já existe!");
25         }
26
27         String encryptedPassword = new BCryptPasswordEncoder().encode(password);
28
29         UserRole roleEnum;
30         try {
31             roleEnum = UserRole.valueOf(roleStr.toUpperCase());
32         } catch (IllegalArgumentException e) {
33             throw new RuntimeException("Role inválido! Use ADMIN ou USER.");
34         }
35
36         User newUser = new User(login, encryptedPassword, roleEnum);
37         repository.save(newUser);
38     }
39
40     public User findByLogin(String login) { return repository.findByUsername(login); // aqui também }
41 }
```

O que é essa classe?

- É um **service** do Spring (`@Service`), ou seja, uma classe que contém **regras de negócio** relacionadas à autenticação/autorização.
 - Ela **não fala diretamente com o banco**, mas usa o `UserRepository` para isso.
-

Vamos entender os métodos:

1. Dependência (`UserRepository`)

java

 Copiar  Editar

```
private final UserRepository repository;

public AuthorizationService(UserRepository repository) {
    this.repository = repository;
}
```

- O Spring injeta automaticamente (`Injeção de Dependência`) o `UserRepository` aqui.
- Assim, esse service pode consultar e salvar usuários no banco.

2. Verifica se o usuário existe

java

 Copiar  Edit

```
public boolean userExists(String login) {  
    return repository.findByUsername(login) != null;  
}
```

- Ele chama o método do repositório (`findByUsername`) para ver se já existe um usuário com aquele login.
- Retorna `true` se o usuário já existe, `false` caso contrário.

➡ O que ele faz aqui:

1. **Checa se o login já existe** → se sim, lança erro.
2. **Criptografa a senha** com `BCryptPasswordEncoder`.
 - Isso garante que a senha não fique salva em texto puro no banco.
3. **Converte a role recebida (String) em Enum** (`ADMIN` ou `USER`).
 - Se a role não for válida, lança exceção.
4. **Cria um novo** `User` e salva no banco.

4. Busca usuário pelo login

java

```
public User findByLogin(String login) {  
    return repository.findByUsername(login);  
}
```

- Método simples: só chama o `findByUsername` do `UserRepository` e retorna o usuário correspondente.

3. Registra um novo usuário

java

```
public void registerUser(String login, String password, String roleStr) {  
    if (userExists(login)) {  
        throw new RuntimeException("Usuário já existe!");  
    }  
  
    String encryptedPassword = new BCryptPasswordEncoder().encode(password);  
  
    UserRole roleEnum;  
    try {  
        roleEnum = UserRole.valueOf(roleStr.toUpperCase());  
    } catch (IllegalArgumentException e) {  
        throw new RuntimeException("Role inválido! Use ADMIN ou USER.");  
    }  
  
    User newUser = new User(login, encryptedPassword, roleEnum);  
    repository.save(newUser);  
}
```

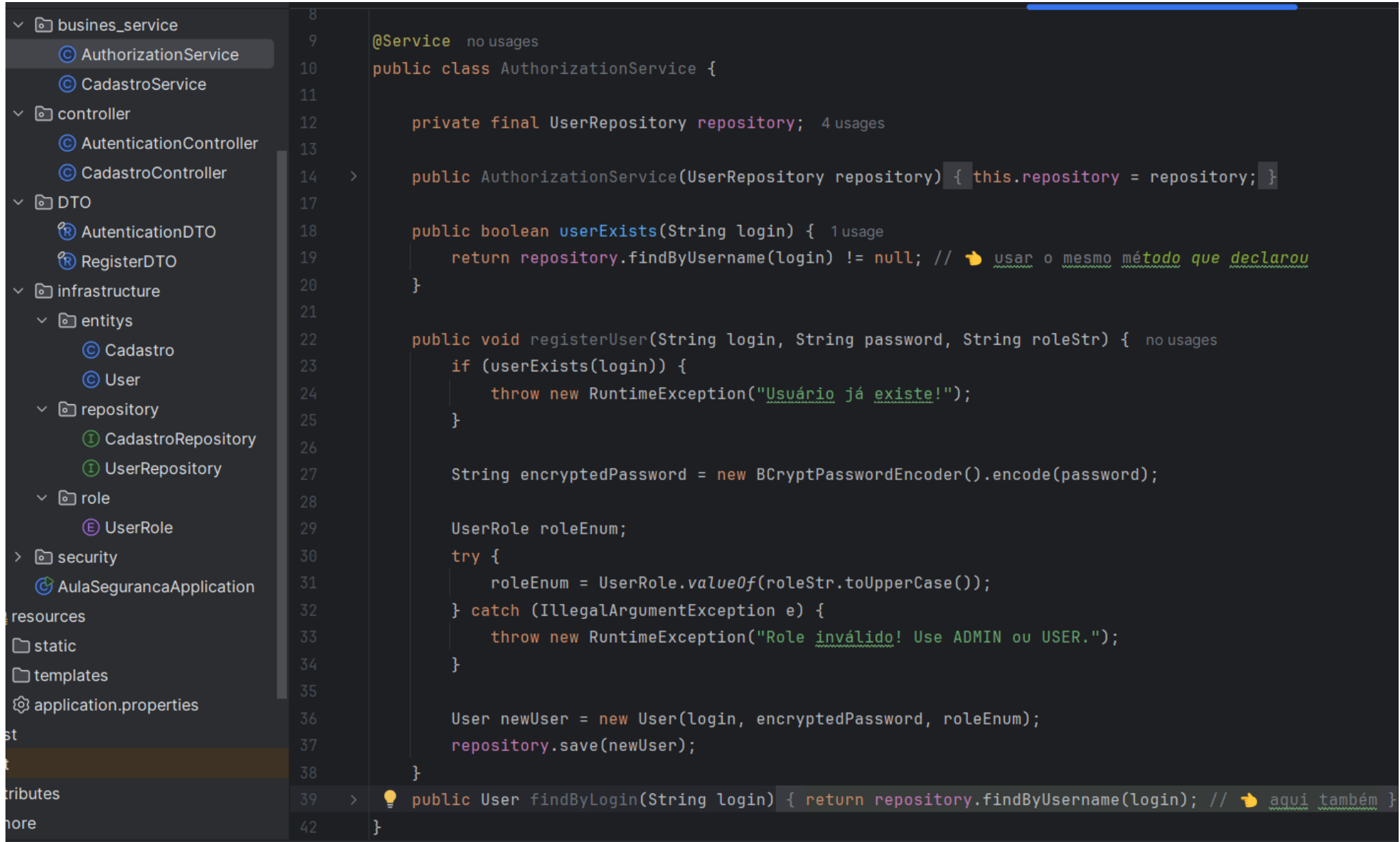


Em resumo:

- `userExists` → verifica se existe login.
- `registerUser` → cria novo usuário (com senha criptografada e role válida).
- `findByLogin` → busca usuário pelo login.

Esse `AuthorizationService` serve como **camada intermediária** entre o controller e o repositório, centralizando a lógica de segurança.

AuthorizationService



The screenshot shows an IDE with a project explorer on the left and a code editor on the right. The project explorer displays a hierarchical structure of the project, including folders like `business_service`, `controller`, `DTO`, `infrastructure`, `entitys`, `repository`, `role`, `security`, `resources`, `static`, `templates`, and `application.properties`. The `AuthorizationService` class is selected under the `business_service` folder.

The code editor displays the implementation of the `AuthorizationService` class. The code is as follows:

```
8
9 @Service no usages
10 public class AuthorizationService {
11
12     private final UserRepository repository; 4 usages
13
14     public AuthorizationService(UserRepository repository) { this.repository = repository; }
15
16
17     public boolean userExists(String login) { 1 usage
18         return repository.findByUsername(login) != null; // usar o mesmo método que declarou
19     }
20
21
22     public void registerUser(String login, String password, String roleStr) { no usages
23         if (userExists(login)) {
24             throw new RuntimeException("Usuário já existe!");
25         }
26
27         String encryptedPassword = new BCryptPasswordEncoder().encode(password);
28
29         UserRole roleEnum;
30         try {
31             roleEnum = UserRole.valueOf(roleStr.toUpperCase());
32         } catch (IllegalArgumentException e) {
33             throw new RuntimeException("Role inválido! Use ADMIN ou USER.");
34         }
35
36         User newUser = new User(login, encryptedPassword, roleEnum);
37         repository.save(newUser);
38     }
39
40     public User findByLogin(String login) { return repository.findByUsername(login); // aqui também }
41
42 }
```

Classe `AuthorizationService`

Essa classe é um **serviço** responsável por **regras de negócio de autenticação/autorização** (criar usuário, verificar se existe, etc).

Ela conversa com o banco de dados através do `UserRepository`.

◆ Dependência do repositório

java

 Copiar  Editar

```
private final UserRepository repository;

public AuthorizationService(UserRepository repository) {
    this.repository = repository;
}
```

- `UserRepository` é injetado no serviço.
- Assim, a classe consegue **buscar e salvar usuários** no banco.

◆ Método `userExists`

java

```
public boolean userExists(String login) {  
    return repository.findByUsername(login) != null;  
}
```

- Verifica se já existe um usuário com esse login.
- O método `findByUsername` vem do **repositório**, que consulta no banco.
- Se retornar `null`, o usuário **não existe**.

◆ Método `registerUser`

java

```
public void registerUser(String login, String password, String roleStr) {  
    if (userExists(login)) {  
        throw new RuntimeException("Usuário já existe!");  
    }  
  
    String encryptedPassword = new BCryptPasswordEncoder().encode(password);  
  
    UserRole roleEnum;  
    try {  
        roleEnum = UserRole.valueOf(roleStr.toUpperCase());  
    } catch (IllegalArgumentException e) {  
        throw new RuntimeException("Role inválido! Use ADMIN ou USER.");  
    }  
  
    User newUser = new User(login, encryptedPassword, roleEnum);  
    repository.save(newUser);  
}
```

Aqui está o processo de **registro de usuário**:

1. **Verifica se já existe usuário com o mesmo login.**
 - Se existir → lança exceção.
2. **Criptografa a senha** com `BCryptPasswordEncoder`.
 - Isso garante que a senha **não seja salva em texto puro** no banco.
 - Mesmo o DBA não consegue ver a senha real.
3. **Valida a role** (`ADMIN` ou `USER`).
 - Converte a string para o `enum UserRole`.
 - Se o valor for inválido → lança exceção.
4. **Cria um novo usuário** com login, senha criptografada e role.
5. **Salva no banco** com `repository.save(newUser)`.

◆ Método findByLogin

java

```
public User findByLogin(String login) {  
    return repository.findByUsername(login);  
}
```

- Serve para **buscar um usuário no banco pelo login**.
- Vai ser útil na hora de autenticar ou carregar dados do usuário.

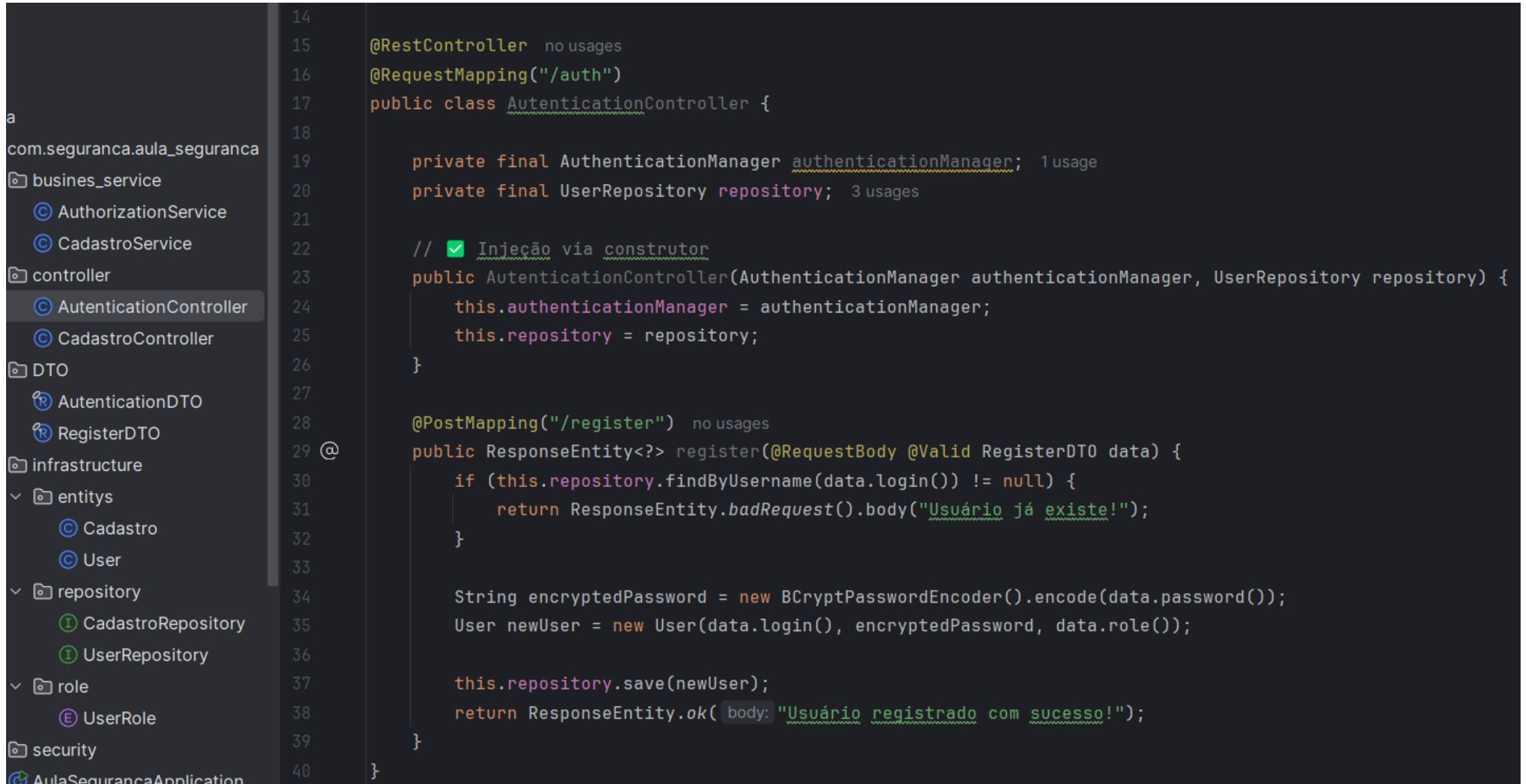
Resumindo

- `userExists` → verifica se já tem o usuário.
- `registerUser` → registra um novo usuário no banco (com senha criptografada + role validada).
- `findByLogin` → busca usuário no banco.

👉 Esse `AuthorizationService` é a **ponte entre o banco e a lógica de autenticação/registro**.

👉 Ele garante segurança (criptografia de senha) e consistência (não deixar duplicar login e role inválida).

AuthenticationController



```
14
15 @RestController no usages
16 @RequestMapping("/auth")
17 public class AuthenticationController {
18
19     private final AuthenticationManager authenticationManager; 1 usage
20     private final UserRepository repository; 3 usages
21
22     // ✅ Injeção via construtor
23     public AuthenticationController(AuthenticationManager authenticationManager, UserRepository repository) {
24         this.authenticationManager = authenticationManager;
25         this.repository = repository;
26     }
27
28     @PostMapping("/register") no usages
29     @
30     public ResponseEntity<?> register(@RequestBody @Valid RegisterDTO data) {
31         if (this.repository.findByUsername(data.login()) != null) {
32             return ResponseEntity.badRequest().body("Usuário já existe!");
33         }
34
35         String encryptedPassword = new BCryptPasswordEncoder().encode(data.password());
36         User newUser = new User(data.login(), encryptedPassword, data.role());
37
38         this.repository.save(newUser);
39         return ResponseEntity.ok( body: "Usuário registrado com sucesso!");
40     }
}
```

📌 O que é essa classe?

É um **controlador REST** do Spring Boot.

Ou seja, uma classe que expõe **endpoints HTTP** para o cliente (Postman, Angular, etc.) poder se comunicar com o sistema.

🔑 Anotações principais

- `@RestController` → indica que a classe é um controller REST.
(Tudo que ela retorna será convertido em JSON ou texto automaticamente).
- `@RequestMapping("/auth")` → significa que todos os endpoints dentro desta classe começam com `/auth`.

Exemplo:

- `POST /auth/register`

🧩 Dependências injetadas

java

📄 Copiar ✎ Editar

```
private final AuthenticationManager authenticationManager;  
private final UserRepository repository;
```

- `AuthenticationManager` → é usado para autenticação (login), embora aqui só esteja sendo injetado, mas ainda não usado.
- `UserRepository` → interface que acessa o banco (via JPA), permitindo **buscar usuários** ou **salvar novos usuários**.

Eles são passados pelo construtor, prática recomendada chamada **injeção de dependência via construtor**.

Endpoint de registro

java

 Copiar  Editar

```
@PostMapping("/register")  
public ResponseEntity<?> register(@RequestBody @Valid RegisterDTO data) { ... }
```

- `@PostMapping("/register")` → define que esse método responde ao **POST /auth/register**.
- `@RequestBody` → indica que os dados virão no corpo da requisição em formato JSON.
- `@Valid` → garante que as validações do DTO serão aplicadas (ex: não pode vir vazio, se você usar anotações como `@NotBlank`).

Lógica do método

1. Verifica se o usuário já existe

java

```
if (this.repository.findByUsername(data.login()) != null) {  
    return ResponseEntity.badRequest().body("Usuário já existe!");  
}
```

- Ele consulta no banco pelo login (username).
- Se já existir, retorna `400 Bad Request`.

2. Criptografa a senha

java



```
String encryptedPassword = new BCryptPasswordEncoder().encode(data.password());
```

- Usa **BCrypt** para proteger a senha antes de salvar.
 - Isso é fundamental, pois **nunca se deve salvar senha em texto puro** no banco.
-

3. Cria um novo usuário

java



```
User newUser = new User(data.login(), encryptedPassword, data.role());
```

- Monta um objeto `User` com:
 - login
 - senha criptografada
 - role (perfil, ex: ADMIN ou USER)

4. Salva no banco

```
java  
  
this.repository.save(newUser);
```

5. Retorna resposta de sucesso

```
java  
  
return ResponseEntity.ok("Usuário registrado com sucesso!");
```

⚡ Fluxo resumido

1. Cliente manda JSON:

```
json  
  
{  
  "login": "joao",  
  "password": "123456",  
  "role": "USER"  
}
```

2. Controller recebe `RegisterDTO`.

3. Verifica se usuário existe.

4. Criptografa senha.

5. Salva no banco.

6. Retorna **200 OK** → "Usuário registrado com sucesso!"

JWT Tokens

O que é JWT?

JWT = JSON Web Token.

É um **token** (um texto codificado) que o backend gera quando o usuário faz login com sucesso. Esse token é devolvido para o cliente (ex: navegador ou app mobile) e precisa ser enviado junto em cada requisição protegida.

Exemplo de um JWT:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
```

```
eyJzdWIiOiJqb2FvIiwicm9sZSI6IkpFETU1OiwiaXNjaXNzMDUyMDAwfQ.
```

```
2PzKlgW9RXr1PzMG8sKgfk0xLzZs3s5Xq4sYm2n6fsI
```

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.

eyJzdWIiOiJqb2FvIiwicm9sZSI6IkFETU10IiwiaXhwIjoxNzMwODUyMDAwfQ.

2PzK1gW9RXr1PzMG8sKgfk0xLzZs3s5Xq4sYm2n6fsI

Ele tem 3 partes:

1. **Header** → Algoritmo de criptografia usado.
2. **Payload** → Informações do usuário (ex: login, role, expiração).
3. **Signature** → Garantia de que não foi alterado.

O que são Roles?

Roles são **papéis** que o usuário pode ter no sistema, definidos na sua entidade `User`.

No seu projeto, você já criou o `UserRole` com, por exemplo:

```
java
```

```
public enum UserRole {  
    ADMIN("admin"),  
    USER("user");  
}
```

Isso significa:

- Um **ADMIN** pode ter acesso a endpoints mais sensíveis (ex: cadastrar usuários).
- Um **USER** pode ter acesso apenas a endpoints comuns.

⚙️ Como se aplica neste projeto?

1. Registro de Usuário

Você já tem isso no `AuthenticationController`.

Quando alguém se registra:

- Senha é criptografada com `BCryptPasswordEncoder`.
- Usuário é salvo com seu `role` (ADMIN ou USER).

2. Login

Você vai criar um endpoint `/login` que:

- Verifica login/senha no banco.
- Se ok → gera um JWT contendo `login` + `role`.
- Retorna o JWT pro cliente.

3. Requisições Seguras

O cliente (frontend/app) precisa enviar o token em **cada requisição**:

```
http
```

```
GET /users
```

```
Authorization: Bearer eyJh...
```

4. Validação do Token

No backend:

- Um filtro do Spring Security intercepta a requisição.
- Decodifica o token.
- Verifica se é válido (assinatura + expiração).
- Se válido → carrega o usuário e suas **roles** no contexto de segurança.

5. Controle de Acesso

No seu Controller ou Service você pode restringir:

java

```
@PreAuthorize("hasRole('ADMIN')")
@GetMapping("/admin")
public String adminOnly() {
    return "Apenas admins acessam!";
}
```

Fluxo Final do Projeto com JWT + Roles

1. Usuário se registra (`/auth/register`).
2. Usuário faz login (`/auth/login`) → recebe um JWT válido.
3. Cliente guarda esse token (no LocalStorage, Cookie ou Header).
4. Em cada requisição protegida → envia `Authorization: Bearer <TOKEN>`.
5. Backend valida token → descobre quem é o usuário e seu papel (role).
6. Se autorizado → libera acesso.
Se não → retorna **403 Forbidden**.

 Resumindo:

- **JWT** = garante *quem é o usuário* (identidade).
- **Roles** = garante *o que ele pode fazer* (permissão).